

Verilog Language and Application

Week 8 Lab Document

Lab Manual

Revision 1.0

© 1990-2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;

- The publication may not be modified in any way;

- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents

Verilog Language and Application

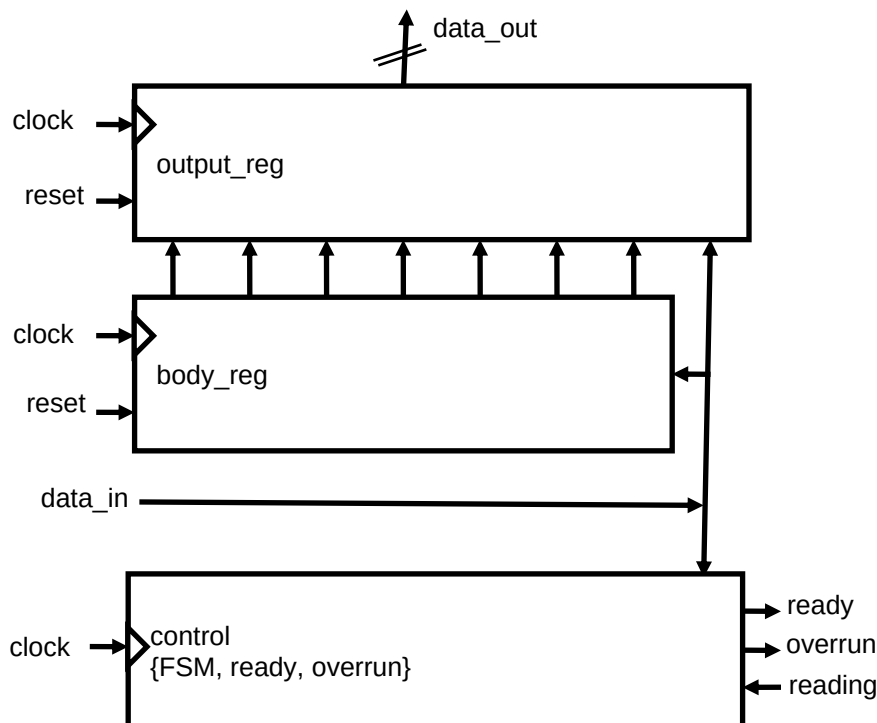
Lab 1	Coding State Machines in Multiple Styles	5
	Specifications.....	5
	Designing a Finite State Machine	6
	Verifying the Finite State Machine Design.....	6
Lab 2	Coding a Serial-to-Parallel Interface Receiver.....	9
	Specifications.....	9
	Designing a Serial-to-Parallel Interface	10
	Verifying the Serial-to-Parallel Interface	10
Lab 3	Adding System Tasks and System Functions to a Beverage Dispenser Design	13
	Specifications.....	13
	Designing a Beverage Dispenser FSM Design	14
	Verifying the Serial-to-Parallel	15

Lab 1 Coding State Machines in Multiple Styles

Objective: To code state machines in different styles for synthesis.

In this lab, you code the serial-to-parallel interface receiver as an FSM in different styles. Instead of shifting the matching character, the FSM steps to the next state upon receiving a correct match bit and returns to a previous state while receiving an incorrect match bit.

Read the specification first and then follow the instructions in this lab. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- ◆ `data_in`, `reading`, `clk`, and `reset` are single-bit input signals.
- ◆ `data_out` is an 8-bit output signal, whereas `ready` and `overrun` are 1-bit output.
- ◆ The design is clocked on the rising edge of `clk`.
- ◆ Every serial bit transmission has a preamble that needs to be sent before sending the character. For this lab, the preamble value is fixed to `8'hA5`.
- ◆ After the preamble, the `ready` signal will be high.

Designing a Finite State Machine

1. Change to the `lab1-fsm` directory and examine the files.
2. Write down the following three FSM styles.
 - a. A single sequential block (as the `rcvr.v` file partially codes).
 - b. Move the “next state” encoding to a separate combinational block.
 - c. Also, move all non-FSM registers to a separate sequential block.
3. In your favorite editor, modify the `rcvr.v` file to comply with the coding style.

Verifying the Finite State Machine Design

1. Verify the RTL model using the following commands with Xcelium™.

```
xrun rcvr.v rcvr_test.v (Batch Mode)
```

2. Verify that you see the following message.

```
Message sending: I Love Verilog
At time 35: Put character "I"
At time 39: Got character "I"
At time 77: Put character " "
At time 95: Got character " "
At time 117: Put character "L"
At time 127: Got character "L"
At time 153: Put character "o"
At time 175: Got character "o"
At time 193: Put character "v"
At time 207: Got character "v"
At time 231: Put character "e"
At time 257: Got character "e"
At time 275: Put character " "
At time 297: Got character " "
At time 309: Put character "V"
At time 313: Got character "V"
At time 349: Put character "e"
At time 365: Got character "e"
At time 391: Put character "r"
```

```
At time 423: Got character "r"
At time 425: Put character "i"
At time 433: Got character "i"
At time 459: Put character "l"
At time 487: Got character "l"
At time 495: Put character "o"
At time 507: Got character "o"
At time 537: Put character "g"
At time 563: Got character "g"
Message received: I Love Verilog
TEST DONE
```

3. Correct your model until it passes the test.
4. When all models pass their test, synthesize the RTL model using the Genus™ Synthesis Solution.

```
genus -f genus_shell.tcl
```
5. Check to see that the *synthesis succeeded*. Correct your model until it is synthesized.
6. Verify the gate-level model using the following commands with Xcelium.

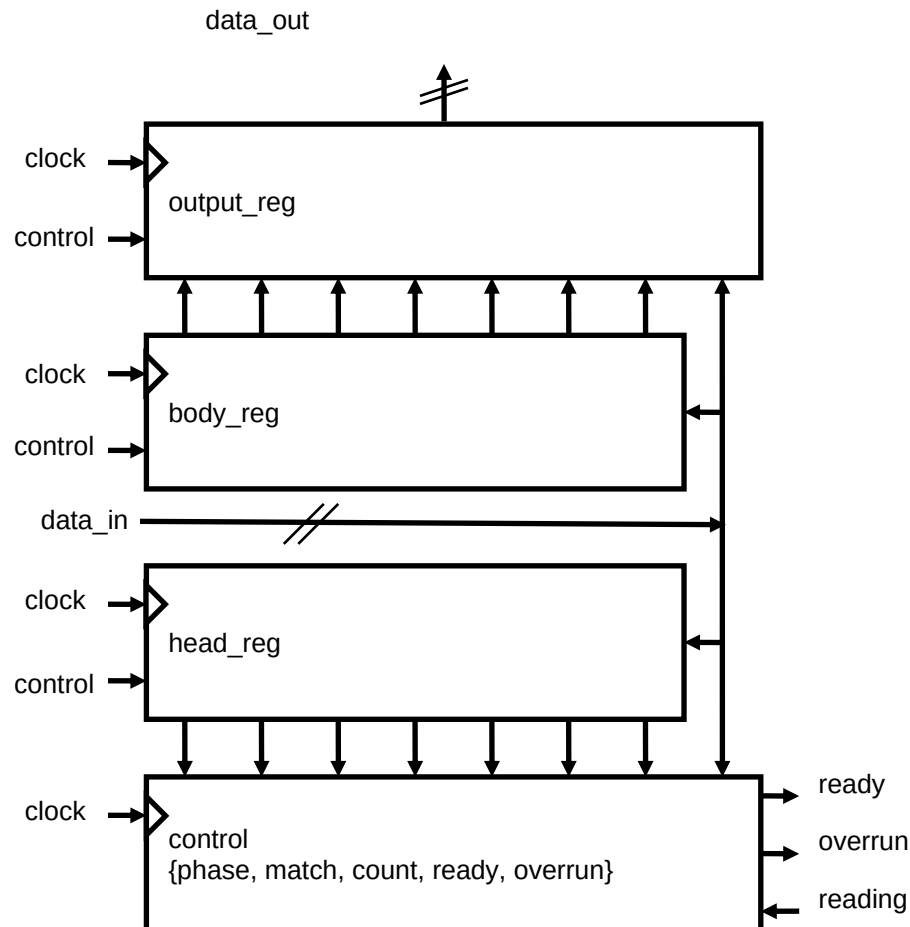
```
xrun rcvr.vg rcvr_test.v -v ../tutorial.v -vlogext vg
```
7. Check to see the messages `I Love Verilog` and `TEST PASSED` as above. If the model fails its test, then you can ask the instructor for help or try again after you study the lecture module *Avoiding Simulation Mismatches*.
8. Correct your design as needed.



Lab 2 Coding a Serial-to-Parallel Interface Receiver

Objective: To code a moderately difficult Verilog RTL design for synthesis.

In this lab, you code a moderately difficult Verilog design for synthesis. Read the specification first and then follow the instructions in the lab section. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- ◆ `data_in`, `reading`, `clk`, and `reset` are single-bit input signals.
- ◆ `data_out` is an 8-bit output signal, whereas `ready` and `overrun` are 1-bit output.
- ◆ The design is clocked on the rising edge of `clk`.
- ◆ Every serial bit transmission has a preamble that needs to be sent before sending the character. For this lab, the preamble value is fixed to `8'hA5`.

- ◆ After the preamble, the `ready` signal will be high.

Designing a Serial-to-Parallel Interface

1. Change to the `lab2-rcvr` directory and examine the following files.

<code>rcvr.v</code>	Receiver model (incomplete)
<code>rcvr_test.v</code>	Receiver test
<code>genus_shell.tcl</code>	Synthesis script

- a. The receiver searches the serial input stream for a match character, and when found, loads the next serial input character to the output buffer, asserts a “ready” flag, and immediately searches for the next match. Modeling the individual parts of the receiver is like modeling individual components.
2. In your favorite editor, modify the `rcvr.v` file to complete the receiver definition.

Verifying the Serial-to-Parallel Interface

1. Verify the RTL model using the following commands with Xcelium™.

```
xrun rcvr.v rcvr_test.v
```

2. You should see the following.

```
Message sending:  I Love Verilog
At time 35: Put character "I"
At time 39: Got character "I"
At time 77: Put character " "
At time 95: Got character " "
At time 117: Put character "L"
At time 127: Got character "L"
At time 153: Put character "o"
At time 175: Got character "o"
At time 193: Put character "v"
At time 207: Got character "v"
At time 231: Put character "e"
At time 257: Got character "e"
At time 275: Put character " "
At time 297: Got character " "
```

```
At time 309: Put character "V"
At time 313: Got character "V"
At time 349: Put character "e"
At time 365: Got character "e"
At time 391: Put character "r"
At time 423: Got character "r"
At time 425: Put character "i"
At time 433: Got character "i"
At time 459: Put character "l"
At time 487: Got character "l"
At time 495: Put character "o"
At time 507: Got character "o"
At time 537: Put character "g"
At time 563: Got character "g"
Message received: I Love Verilog
TEST DONE
```

Correct your model until it passes the test.

3. When all models pass their test, synthesize the RTL model by entering:

```
genus -f genus_shell.tcl
```

Check to see that the *synthesis succeeded*.

Correct your model until it can synthesize.

4. Verify the gate-level model using the following commands with Xcelium.

```
xrun rcvr.vg rcvr_test.v -v ../tutorial.v -vlogext vg
```

Check to see the messages I Love Verilog and TEST PASSED similar to, as shown above.

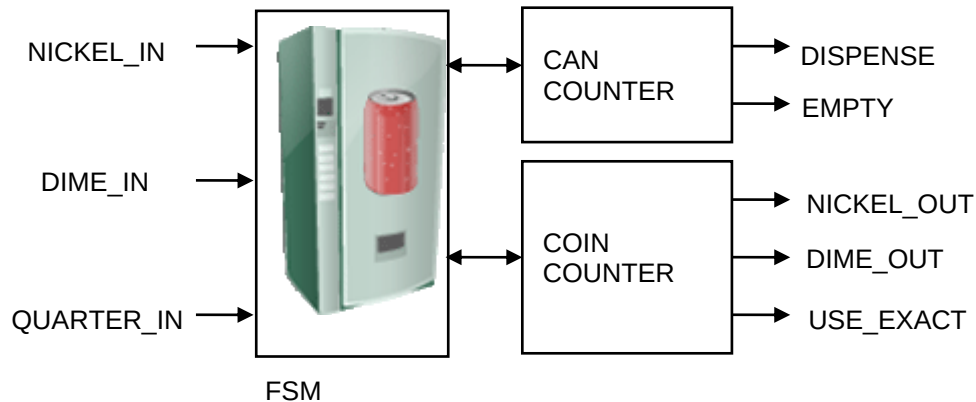
If the model fails its test, then you can ask the instructor for help or try again after you study the lecture module *Avoiding Simulation Mismatches*.



Lab 3 Adding System Tasks and System Functions to a Beverage Dispenser Design

Objective: To add system tasks and system functions to support debugging efforts.

The lab model is a beverage dispenser (drink machine). Read the specification first and then follow the instructions in the lab section. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

1. The drink machine operates on the positive clock edge and has a synchronous high-active reset.
2. After resetting the machine, you load it with coins and cans.
3. Then, as you insert `NICKELS`, `DIMES`, and `QUARTERS`, at 50 cents or above, it dispenses a can and attempts to issue your change (if any).
4. This drink machine has no coin return feature and inserted coins are not available later as change.
5. If the `EMPTY` signal is true, then you lose any coins you insert.
6. If the `USE_EXACT` signal is true, then you might find yourself short-changed.
7. As the machine is currently defined, it does not attempt to issue lower-denomination coins if higher-denomination coins are not available.

8. **Drink Machine State Table:** Models the behavior for accepting coins and dispensing drinks:

- Amount of money that the user has deposited so far defines the current state.
- The type of coin that the user deposits determines the machine's next state.

Current State	Transition Value	Next State (Change)
idle 4'd0	nickel_in dime_in quarter_in	five ten twenty_five
five 4'd1	nickel_in dime_in quarter_in	ten fifteen thirty
ten 4'd2	nickel_in dime_in quarter_in	fifteen twenty thirty_five
fifteen 4'd3	nickel_in dime_in quarter_in	twenty twenty_five forty
twenty 4'd4	nickel_in dime_in quarter_in	twenty_five thirty forty_five
twenty_five 4'd5	nickel_in dime_in quarter_in	thirty thirty_five idle
thirty 4'd6	nickel_in dime_in quarter_in	thirty_five forty idle {nickel_out}
thirty_five 4'd7	nickel_in dime_in quarter_in	forty forty_five idle {dime_out}
forty 4'd8	nickel_in dime_in quarter_in	forty_five idle idle {nickel_out, dime_out}
forty_five 4'd9	nickel_in dime_in quarter_in	idle idle {nickel_out} idle {two_dime_out}

Designing a Beverage Dispenser FSM Design

9. Change to the lab3-dkm directory and examine the files provided.

dkm.v	DUT
test.v	Test(incomplete)

Note: The test developer developed the test in a top-down manner, determining and coding top-level tasks first, then subtasks, and then more subtasks. Only the lowest-level tasks actually interact with the machine. You can also adopt this approach to test development.

10. Design and test are given; you have to modify the test file as instructed.

- a. Improve the *expect()* task so that it displays an error message and displays each machine output that is erroneous.
- b. Display the expected and actual values in *expect()* task.
- c. Display the simulation time and terminate the simulation.
- d. Add a monitor code in the test file with the following functionality.
 - i. Monitors drink machine inputs and outputs to a disk file. The monitor should print the simulation time of each signal change and should use the %t formatter so that the printed time value takes a fixed number of columns.
 - ii. Dumps all top-level drink machine nets and variables into a VCD file.
 - iii. Dumps all drink machine ports to an extended VCD file.
- e. Improve the test procedure so that if it completes, then it displays a happy message and terminates the simulation.

Verifying the Serial-to-Parallel

1. Simulate the design and test using the following commands with Xcelium.

```
xrun dkm.v test.v (Batch Mode)
```

or

```
xrun dkm.v test.v -gui -access +rwc ( GUI Mode)
```

2. On successful simulation, you will see the following display.

```
CANS 1 ; COINS 0,0 ; INSERT 0,0,2
CANS 1 ; COINS 1,2 ; INSERT 0,3,1
CANS 1 ; COINS 1,2 ; INSERT 0,1,2
CANS 1 ; COINS 2,3 ; INSERT 1,1,2
CANS 1 ; COINS 1,4 ; INSERT 1,1,2
TEST PASSED
```

3. If you have sufficient remaining lab time, add the interactive test by adding the **+INTERACTIVE=1** invocation option. The interactive test should add some cans and coins and then, in a loop, repeatedly request the user to insert a coin. Your code can read an input character, discard the remainder of the input line, and, depending upon the character, insert a nickel, dime, or quarter. Your code should check and report the machine outputs after dispensing each can. You may find the following information useful.
 - a. `stdin` is file descriptor `32'h8000_0000`
 - b. `stdout` is file descriptor `32'h8000_0001`
 - c. `reg_8_bit = $fgetc ( file_descriptor ); // gets the one next 8-bit character`
 - d. `error_integer = $fgets ( reg_vector, file_descriptor ); // gets remaining line`
(Ensure that `reg_vector` is sufficiently wide enough to get the entire remaining line.)

Note: You must increase the clock count to accommodate this additional test. Now that your test procedure gracefully terminates the test, you can alternatively replace the *repeat (n)* loop with a *forever* loop.
4. Simulate the design and test and correct the test as needed.
5. Load the VCD file into a graphical simulation analysis environment by entering:
`simvision your_dumpfile_name`
6. Verify that the drink machine operates correctly.

